

Hurricane Harvey Damage Classification Report

Introduction

This project focuses on developing deep learning models to automatically classify building damage from satellite images captured after Hurricane Harvey. By comparing multiple model architectures and evaluating their performance on a curated dataset, the project demonstrates how modern computer vision methods can assist in post-disaster assessment. The work included data preprocessing, model design and training, quantitative evaluation, and inference server deployment in a reproducible environment.

Data Preparation

The dataset consisted of satellite images labeled as either *damaged* or *non-damaged* buildings after Hurricane Harvey. The original images were all JPEGs of size 128×128 pixels, but that resolution was computationally heavy for training and unnecessary for coarse damage detection. To balance computational efficiency with feature visibility, all images were scaled by a factor of $\frac{1}{2}$ to 64×64 pixels.

Each image was then converted to RGB and normalized by dividing pixel values by 255 to map them into $[0, 1]$, ensuring consistent input ranges across all channels. Normalization helped the network converge faster and prevented large gradient updates.

To maintain balanced representation between classes, the dataset contained 14,170 damaged and 7,152 non-damaged samples, which were combined and split 70 / 15 / 15 into training, validation, and test sets (14,925 / 3,198 / 3,199 images). Stratified splitting was used so the damage–no-damage ratios were respected across all subsets.

A helper function automated loading, resizing, and labeling, returning NumPy arrays suitable for TensorFlow. Visual inspections of random samples verified that preprocessing preserved clear visual differences, such as roof discoloration, debris, and structural shadows, between classes.

These preprocessing choices were influenced by practical trade-offs: smaller images reduce training time without majorly sacrificing critical patterns, and normalization stabilizes optimization. Together, these steps established a clean, balanced dataset ready for convolutional modeling.

Model Design and Training

Three different neural network architectures were designed and trained to classify the images: a fully-connected dense ANN, the classic LeNet-5 CNN, and a modified Alternate LeNet-5 architecture.

1. Fully Connected (ANN)

Images were flattened into one-dimensional vectors before feeding them into dense layers (128 → 64 → 1). This design ignores spatial structure and therefore relies purely on pixel-wise intensity patterns. Using ReLU activations and Adam optimization, the model achieved only ≈ 66 % accuracy on the test set. This result was expected, since flattening the images removes important spatial relationships like edges, corners, and textures. Without that context, the network struggles to understand the structure of the image. Still, the dense ANN served as a good baseline to compare the more advanced architectures against.

2. LeNet-5 CNN

The second model implemented a convolutional design modeled after the LeNet-5 architecture. Two convolutional layers (6 and 16 filters) were followed by average-pooling layers, then three dense layers (120 → 84 → 1). Compared to *tanh* activations, ReLU significantly improved convergence and accuracy in TensorFlow for this model. Trained for 15 epochs with a batch size of 32, the LeNet-5 CNN achieved a test accuracy of 93.1 %. Unlike the dense ANN, this network first learned spatial patterns through convolution and pooling before flattening, allowing it to effectively extract low-level visual features like edges, contrast, and texture patches. This demonstrated the clear advantage of spatial feature learning over flattened pixel inputs.

3. Alternate LeNet-5 CNN

The best-performing model was a modified version of LeNet-5 that replaced average pooling with max pooling and increased the number of filters to 32 and 64 in the two convolutional layers. Max pooling improved feature selectivity by preserving high-activation regions corresponding to key damage indicators. This architecture achieved a test accuracy of 93.6 %, slightly outperforming the standard LeNet-5. The improvement illustrates how filter count and pooling choice impact representational power. Despite its higher parameter count, training remained stable, and validation loss showed minimal overfitting, indicating strong generalization across unseen test images.

Overall, network depth, activation functions, and pooling methods were iteratively tuned for speed and accuracy. ReLU activations, Adam optimization, and moderate filter sizes offered the best balance between training time and performance stability.

Model Evaluation

Performance Comparison	
Model	Test Accuracy
Dense ANN	0.6646
LeNet-5 CNN	0.9312
Alternate LeNet-5 CNN	0.9359

The alternate LeNet-5 model clearly outperformed the others. The $\approx 27\%$ accuracy jump from the dense ANN to the CNN architecture demonstrates that learning spatial patterns and structure in the images is essential for this task. The difference between the two CNNs (highlighted by a 0.4% gain) shows the effect of max pooling and increased filter capacity on sharper feature capture.

Confidence in Results

Model performance was consistent across training and validation phases, with validation accuracy tracking closely to training accuracy ($\sim 93\%$) and no significant divergence in loss curves. This suggests that overfitting was minimal and that the model generalizes well to unseen data. Given the large sample size (over 21k images) and strong accuracy on a held-out test set, we can be confident that the model will perform reliably when deployed on similar aerial imagery datasets. Additionally, misclassified cases tended to involve borderline images with partial damage or obscured features, situations that are generally more complex or ambiguous. This supports the idea that the model's remaining errors stem from inherent ambiguity in the dataset rather than from systematic bias in its learning process.

Model Deployment and Inference

To make our best model usable beyond the notebook environment, we deployed it to a lightweight inference server running inside a Docker container. This configuration allows users to interact with the model using HTTP requests. We automated the deployment of our server by using a Makefile. Running `make` will automatically bring existing containers down and then spin up a new container.

Our inference server uses a Flask API to allow users to access endpoints using 2 types of HTTP requests.

1. (GET) **/summary**
 - Provides a summary of the model's metadata to the user.
 - Example: `curl localhost:5000/summary`
2. (POST) **/inference**
 - Allows the user to provide their own image to the inference server, so the model can predict whether or not the building in the picture is damaged or not.
 - Example: `curl -X POST -F "image=@<path_to_house_img>" localhost:5000/inference`

Example outputs and instructions for using the grader are found in the [README](#).

Summary

In conclusion, we developed 3 neural networks to detect whether satellite images contained damaged or non-damaged buildings after Hurricane Harvey. We evaluated each and deployed our best model to an inference server, which users can interact with via a web API. The results showed that CNNs significantly outperform vanilla ANNs in image recognition. Overall, this project highlights many stages of the machine learning lifecycle, from data pre-processing to model training evaluation to inference server deployment.